

Secure Recipes GRETA 92

Projet final formation GRETA 92

Developpement securise d'un site de recettes de cuisine

Otmane Aiboud

PHP · HTML · JavaScript · Tailwind CSS · MySQL

Projet final formation GRETA 92

Developpement securise d'un site de recettes de cuisine

Otmane Aiboud

Stack : PHP, HTML, JavaScript, Tailwind CSS, MySQL

Theme : securite applicative web

Sommaire

1. Contexte
2. Architecture
3. Fonctionnalites
4. Securite applicative
5. Tests de securite realises
6. Conclusion

1. Contexte

Secure Recipes GRETA 92 est un site de recettes de cuisine concu comme une application reelle exposee a Internet. Le public peut consulter les recettes. Les administrateurs authentifies peuvent gerer les recettes et les comptes administrateurs. Le projet applique les protections fondamentales contre XSS, injection SQL, CSRF, brute force et upload de fichiers dangereux.

2. Architecture

- ``public/`` : accueil, detail recette, login, logout, presentation, assets.
- ``admin/`` : dashboard, CRUD recettes, CRUD administrateurs.
- ``public/admin/`` : wrappers compatibles avec une racine web ``public/`` pour exposer les URLs ``/admin/...``.
- ``app/config/`` : configuration applicative et connexion PDO.
- ``app/security/`` : sessions, authentication, CSRF, CSP, brute force, upload.
- ``app/repositories/`` : requetes SQL preparees.
- ``app/validation/`` : validation serveur.
- ``docs/`` : rapport Markdown et PDF.
- ``database.sql`` : schema MySQL, donnees de demonstration et admin initial.

3. Fonctionnalites

- Accueil public avec hero, badges securite et liste des recettes.
- Detail recette avec titre, image, description, ingredients et preparation.
- Connexion admin sans inscription publique.
- Dashboard admin avec compteurs et tentatives echouees recentes.

- CRUD recettes avec upload image.
- CRUD administrateurs avec hachage des mots de passe.
- Page presentation type PowerPoint integree.

4. Securite applicative

A. Authentification et hachage des mots de passe

Technique : `password_hash()` et `password_verify()`.

Menace : vol ou lecture directe des mots de passe si la base est compromise.

Solution appliquee : les mots de passe admins sont haches. Le login compare le mot de passe saisi avec le hash stocke.

Fichiers concernes : `admin/admins/create.php`, `admin/admins/edit.php`, `public/login.php`.

Extrait reel :

```
$repo->create([
    'username' => $data['username'],
    'email' => $data['email'],
    'password_hash' => password_hash($data['password'], PASSWORD_DEFAULT),
]);

$valid = $admin && password_verify($password, (string) $admin['password_hash']);
```

Explication : `password_hash()` choisit un algorithme adapte par default. `password_verify()` evite de comparer manuellement le mot de passe.

Limite restante : les identifiants de demonstration doivent etre changes en production.

B. Sessions securisees

Technique : cookies securises et regeneration d'identifiant de session.

Menace : fixation de session et vol de cookie.

Solution appliquee : session `HttpOnly`, `SameSite=Lax`, `Secure` si HTTPS et regeneration apres connexion.

Fichiers concernes : `app/security/auth.php`.

Extrait reel :

```
session_set_cookie_params([
    'lifetime' => 0,
    'path' => '/',
    'domain' => '',
    'secure' => $https,
    'httponly' => true,
    'samesite' => 'Lax',
]);

function login_admin(array $admin): void
```

```
{
    session_regenerate_id(true);
    $_SESSION['admin_id'] = (int) $admin['id'];
}
```

Limite restante : HTTPS doit être forcé au niveau hébergeur en production.

C. Protection XSS

Technique : échappement HTML centralisé.

Menace : exécution de JavaScript injecté dans une recette ou un compte admin.

Solution appliquée : toutes les données affichées depuis la base passent par `e()`.

Fichiers concernés : `app/helpers/functions.php`, `public/index.php`, `public/recipe.php`, `admin/\*`.

Extrait réel :

```
function e(mixed $value): string
{
    return htmlspecialchars((string) $value, ENT_QUOTES, 'UTF-8');
}

<h1 class="mt-5 text-4xl font-bold text-white"><?= e($recipe['title']) ?></h1>
```

Limite restante : si un jour du HTML enrichi est autorisé, il faudra ajouter une liste blanche stricte.

D. Content Security Policy

Technique : header CSP centralisé.

Menace : exécution de scripts externes ou injection de contenu actif.

Solution appliquée : CSP `default-src 'self'`, scripts et styles locaux, images locales/data, objets interdits, framing interdit.

Fichiers concernés : `app/security/headers.php`, `app/bootstrap.php`.

Extrait réel :

```
header("Content-Security-Policy: default-src 'self'; script-src 'self'; style-src
'self'; img-src 'self' data:; font-src 'self' data:; object-src 'none'; base-uri 'self';
frame-ancestors 'none'; form-action 'self'");
```

Limite restante : vérifier les headers depuis l'URL finale Railway après déploiement.

E. Protection injection SQL

Technique : PDO prepare/execute.

Menace : modification d'une requête SQL par saisie utilisateur.

Solution appliquee : les repositories utilisent des requetes preparees et des parametres.

Fichiers concernes : `app/repositories/RecipeRepository.php`, `app/repositories/AdminRepository.php`, `app/repositories/LoginAttemptRepository.php`.

Lecture :

```
$stmt = $this->pdo->prepare('SELECT * FROM recipes WHERE slug = :slug LIMIT 1');  
$stmt->execute(['slug' => $slug]);
```

Creation :

```
$stmt = $this->pdo->prepare(  
    'INSERT INTO recipes (title, slug, short_description, description, ingredients,  
    preparation_steps, image_path, created_at, updated_at)  
    VALUES (:title, :slug, :short_description, :description, :ingredients,  
    :preparation_steps, :image_path, NOW(), NOW())'  
);
```

Modification :

```
$stmt = $this->pdo->prepare(  
    'UPDATE admins SET username = :username, email = :email, updated_at = NOW() WHERE id  
    = :id'  
);
```

Suppression :

```
$stmt = $this->pdo->prepare('DELETE FROM recipes WHERE id = :id');  
$stmt->execute(['id' => $id]);
```

Limite restante : utiliser en production un utilisateur MySQL dedie avec droits limites, comme indique dans `database.sql`.

F. Protection CSRF

Technique : token aleatoire en session, champ cache et verification serveur.

Menace : forcer un admin connecte a executer une action sensible.

Solution appliquee : les formulaires de connexion, creation, modification et suppression incluent un token.

Fichiers concernes : `app/security/csrf.php`, `public/login.php`, `admin/recipes/\*`, `admin/admins/\*`.

Extrait reel :

```
function generate_csrf_token(): string  
{  
    if (empty($_SESSION['csrf_token'])) {  
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));  
    }  
  
    return (string) $_SESSION['csrf_token'];  
}  
  
function verify_csrf_token(?string $token): bool  
{  
    return is_string($token)  
        && isset($_SESSION['csrf_token'])  
        && hash_equals((string) $_SESSION['csrf_token'], $token);  
}
```

```
}
```

Limite restante : une rotation de token par formulaire pourrait etre ajoutee pour des exigences plus strictes.

G. Protection brute force

Technique : table `login\_attempts`, comptage des echecs recents.

Menace : tentative automatisee de deviner le mot de passe admin.

Solution appliquee : apres 5 echecs sur 15 minutes par email ou IP, la connexion est temporairement refusee.

Fichiers concernes : `app/security/brute\_force.php`, `app/repositories/LoginAttemptRepository.php`, `public/login.php`.

Extrait reel :

```
const MAX_LOGIN_FAILURES = 5;
const LOGIN_WINDOW_MINUTES = 15;
const LOGIN_BLOCK_MINUTES = 15;

$failures = $repo->countRecentFailures($email, client_ip(), LOGIN_WINDOW_MINUTES);
return $failures >= MAX_LOGIN_FAILURES;
```

Limite restante : un nettoyage automatique des anciennes tentatives peut etre planifie.

H. Upload securise des images

Technique : extension, MIME, taille, nom aleatoire et blocage execution.

Menace : televersement de fichier executable ou fichier deguise.

Solution appliquee : seuls `jpg`, `jpeg`, `png`, `webp` sont acceptes. Le MIME est verifie avec `finfo`. Le nom original n'est jamais reutilise.

Fichiers concernes : `app/security/upload.php`, `public/uploads/recipes/.htaccess`.

Extrait reel :

```
$allowedExtensions = ['jpg', 'jpeg', 'png', 'webp'];
if (!in_array($extension, $allowedExtensions, true)) {
    return ['path' => null, 'error' => 'Extension refusee. Formats acceptes : jpg, jpeg, png, webp.'];
}

$finfo = new finfo(FILEINFO_MIME_TYPE);
$mime = $finfo->file($tmpPath) ?: '';

$filename = bin2hex(random_bytes(16)) . '.' . $extension;
```

Limite restante : stocker hors webroot et servir via script controle serait encore plus robuste.

I. Protection des acces admin

Technique : middleware `require_admin()`.

Menace : acces direct aux pages `/admin` sans authentication.

Solution appliquee : chaque page admin appelle `require_admin()` avant d'afficher le contenu.

Fichiers concernes : `app/security/auth.php`, `admin/*`.

Extrait reel :

```
function require_admin(): void
{
    if (!is_admin_authenticated()) {
        flash('error', 'Acces reserve aux administrateurs authentifies.');
```

```
        redirect('/login.php');
    }
}
```

Limite restante : ajouter un controle serveur web pour refuser l'indexation des URLs admin.

J. Validation des entrees

Technique : validation serveur par type de formulaire.

Menace : donnees incompletes, trop longues ou non conformes.

Solution appliquee : les recettes et admins sont controles avant toute insertion ou modification.

Fichiers concernes : `app/validation/recipe_validation.php`, `app/validation/admin_validation.php`.

Extrait reel :

```
if ($title === '' || mb_strlen($title) > 150) {
    $errors['title'] = 'Le titre est obligatoire et limite a 150 caracteres.';
}

if (!filter_var($email, FILTER_VALIDATE_EMAIL) || mb_strlen($email) > 190) {
    $errors['email'] = 'Email administrateur invalide.';
}
```

Limite restante : ajouter des tests automatises de validation.

5. Tests de securite realises

- Tentative XSS dans titre recette : affichee comme texte avec `htmlspecialchars()`.
- Tentative SQLi dans login : requete preparee, pas de concatenation SQL.
- Suppression recette sans CSRF : refusee par `require_valid_csrf()`.
- Acces `/admin/dashboard.php` sans connexion : redirection login.
- Upload fichier `.php` : extension refusee.
- Upload image trop lourde : limite 2 Mo.
- Plusieurs echecs login : blocage apres 5 echecs recents.
- Mot de passe stocke : hash present dans `database.sql`, aucun mot de passe clair en table.

- CSP presente : header centralise dans `headers.php`.
- Navigation responsive : Tailwind compile localement.

6. Conclusion

Secure Recipes GRETA 92 montre une application PHP/MySQL complete, structuree et securisee. Le projet relie chaque fonctionnalite a une protection concrete et documentee. Il reste volontairement simple cote architecture pour que le jury puisse identifier les mecanismes de securite sans abstraction inutile.